Claude Code Skills for Academic Researchers

A Practical Introduction

Alessandro Spina

UTS Finance Department

April 2026

Roadmap

1. The Problem
2. What Are Skills?
3. How Skills Work
4. Example Skills for Researchers
5. Setting Up Your Skills
6. Practical Advice
7. Summary

1
The Problem

The RA Who Forgets Everything

Every time you start a new Claude Code session, you are working with a **brand-new research assistant** who knows nothing about your way of working.

You end up repeating the same instructions every time.

- `"Use data.table, not tidyverse"`
- `"Compile with pdfLaTeX, not XeLaTeX"`
- `"Always cluster standard errors"`

Key Insight

Skills encode this knowledge **once** — reusable across sessions and projects.

2
What Are Skills?

Skills = An Onboarding Guide for Claude

A Skill is a folder containing instructions, templates, and scripts that Claude Code loads **on-demand** when they are relevant to your task.

Think of it as writing an onboarding guide for a new RA — except the RA is an AI agent, and the guide is a Markdown file.

Key property: Skills are **reusable across sessions and projects**.

Skills vs. CLAUDE.md

Both persist instructions across sessions, but they serve different purposes:

	CLAUDE.md	Skills
Scope	Short conventions	Rich workflows
Loading	Always loaded	On-demand
Files	Single file	Folder + subfiles
Example	`"Use data.table"`	Full patterns + templates

Rule of Thumb

If it is a **few lines** of convention, put it in `CLAUDE.md`.

If it is a **full workflow** with examples, templates, or helper scripts, make it a Skill.

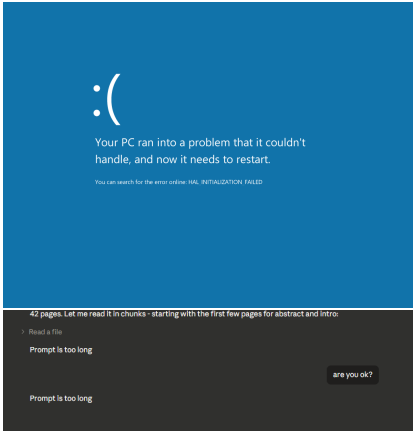
The Context Window Problem

The Inevitable Crash

Every LLM has a finite **context window**. Once it fills up, conversation quality degrades: "context rot".

How to manage it:

- Keep CLAUDE.md concise — practitioners find **~100–150 lines** is a practical limit. Detailed rules go in separate files.
- Use Skills to describe more complex workflows that are only loaded into the context window when triggered.



3
How Skills Work

Anatomy of a Skill

Every Skill is a folder with one required file (the Skill.md) and a few optional subfolders:

```
my-skill/
SKILL.md                          Required
scripts/              Code Claude can execute
references/                 Docs for guidance
assets/                     Templates, themes
```

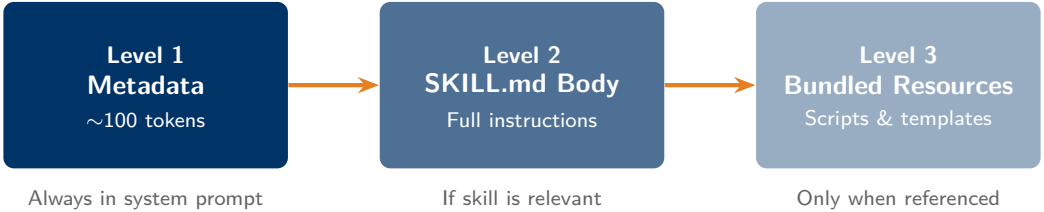
References

Things Claude **reads** to inform its thinking. E.g., style guides, fixest patterns.

Assets

Things Claude **copies or uses directly** in output. E.g., a Beamer preamble, a `.sty` file.

Progressive Disclosure: Claude Only Loads What It Needs



Claude will use the Metadata to check which Skill it should use, and if triggered, only then will load the full instructions + additional resources.

The SKILL.md File

Two parts: **YAML frontmatter** (metadata) and **Markdown body** (instructions).

```
---
name: r-empirical-finance
description: >
Conventions for writing empirical finance R code
with data.table, fixest, arrow, and ggplot2. Use
whenever writing or reviewing R scripts for panel
data, DiD, IV, or regressions.
---
# R Empirical Finance Code Standards
## Package stack
Always prefer data.table (never dplyr/tidyverse)...
```

The Description Is Everything

Claude uses the `description` field to decide whether to load a skill. It tends to **under-trigger**, so be explicit and slightly pushy.

Weak Description

"Helps with R code for research."

Strong Description

*"Conventions for writing empirical finance R code with **data.table, fixest, arrow**, and ggplot2. Use this skill whenever writing, reviewing, or debugging R scripts for panel data, event studies, DiD, IV/2SLS... **even if** the user doesn't explicitly mention these packages."*

Tips

- Include **trigger phrases**: "Use this skill whenever..."
- Name **specific packages** and methods
- Add "**even if** the user doesn't mention X" language

4
Example Skills for Researchers

Skill 1: R Empirical Code Standards

Encodes your preferences so Claude **always** writes R code your way:

1. `data.table` over tidyverse, always
2. `fixest::feols()` for regressions
3. `arrow` for Parquet I/O
4. Clustered SEs by default
5. Auto-warn: missing clustering, duplicates, bad controls

```
references/
fixest-patterns.md
Advanced patterns loaded
only when needed:
- Stepwise estimation
- Split samples
- Lag operators
- IV first-stage diagnostics
```

Skill 2: Structured Paper Review

When you drop a paper PDF into Claude, get a **consistent summary**:

1. One-sentence summary
2. Research question & why it matters
3. **Identification strategy** — method, FEs, clustering
4. Key findings with **actual magnitudes**
5. Robustness, threats, connections to your work

Key Differentiator

The **identification strategy** section is what sets this apart from a generic summary.

Forces Claude to report: exogenous variation, instrument validity, parallel trends — not just "they ran a regression."

Skill 3: Beamer Slide Checker

A five-phase **audit loop** that compiles, inspects, and fixes your slides automatically:

1. Compile `.tex` and parse log for errors
2. Render slides as PNGs, inspect visually
3. Targeted fixes (overflow, layout, images)
4. Re-compile and verify no new issues
5. Summarise all changes made

Design Principle

Prefer **splitting content** across slides over shrinking text.

Readability matters more than slide count.

Diagnostic Script

Generates a `diagnostics.json` with error counts, warnings, and per-slide PNG renders for visual inspection.

Skill 4: Dataset Profiler

A five-phase protocol for **systematically exploring** a new dataset before analysis:

1. Structure discovery (dims, types, IDs)
2. Unit of observation (uniqueness tests)
3. Variable profiling (stats, missingness)
4. Relationships & red flags
5. Compile permanent profile document

Philosophy

"The boring profiling work done here prevents data quality bugs from surfacing **three weeks** into the analysis."

Output

Produces a permanent **Markdown profile doc** stored in the project repo — a reference for the life of the paper.

Skill 5: Skill Writing Guide

A meta-skill that helps you **write better skills**:

1. Correct folder structure (`SKILL.md` required)
2. Write pushy, keyword-rich descriptions
3. Use concrete examples, not abstract rules
4. Include "what NOT to do" sections
5. Split when `SKILL.md` exceeds ~200 lines

The 3-Correction Rule

If you have corrected Claude **3+ times** on the same thing, it belongs in a skill.

This skill helps you capture those corrections **properly**.

Iteration

Test with natural prompts, not "use skill X to..." — if it doesn't trigger naturally, fix the description.

5
Setting Up Your Skills

Where Skills Live

Skills can be installed at different levels depending on how broadly you want them to apply:

Location	Scope	Use case
`~/.claude/skills/`	All projects	Personal conventions
`.claude/skills/`	One project	Paper-specific
Claude.ai Settings	Web UI	Upload as zip

Recommendation

Use **global** for cross-project conventions (R style, Beamer, code audit).

Use **project-level** for paper-specific workflows and data-handling rules.

Recommended Repo Structure

Host your skills in a **GitHub repo**. Clone into `~/.claude/skills/` (on your local device).

```
claude-skills/
r-empirical-finance/
SKILL.md
references/fixest-patterns.md
beamer-slides/
SKILL.md
assets/uts-beamer-preamble.tex
code-audit/ SKILL.md
paper-review/ SKILL.md
```

Structure

Each skill is a **self-contained folder**.
SKILL.md is required.
Subfolders are optional.
Supporting files loaded on demand.

See example:
`https://github.com/aspi6246/Claude-Code-Skills-for-Academics`

6
Practical Advice

What Makes a Good Skill

Do

- Start small — one page, one pain point
- Use **concrete examples**, not abstract rules
- Encode your corrections: 3+ repeats ⇒ skill
- Be explicit about what *not* to do

Don't

- Encode entire methodology in one skill
- Write a "meta-skill" before 3–4 regular skills
- Test with "use skill X to..." — use natural prompts

If you have corrected Claude **3+ times** on the same thing, it belongs in a skill.

Know What Skills Cannot Do

Skills are **reactive**, not scheduled. They fire when Claude decides the skill is relevant.

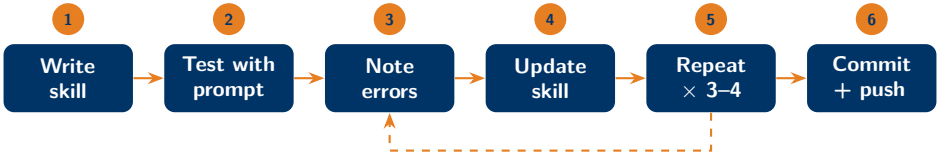
Cannot

- **Run on a cron job** ("check SSRN every Monday")
- **Scrape websites** in the background
- **Maintain state** between sessions

Can

- **Standardise** how Claude analyses a paper
- **Enforce** coding conventions across projects
- **Implement** multi-step review protocols
- **Generate** documents from templates

The Iteration Workflow



Skills are **living documents**. Every time you correct Claude on something that should have been in the skill, go back and add it.

7
Summary

Key Takeaways

1. Skills **encode research conventions** so Claude remembers them
2. They are just **folders with a SKILL.md** file — no special tooling required
3. Start with **2–3 skills** that address your biggest pain points
4. Host in a **GitHub repo**, clone into `~/.claude/skills/`
5. Write **pushy descriptions** so Claude actually triggers them
6. **Iterate** based on real usage — add corrections as you go

Resources

- **Anthropic Skills documentation**
  https://code.claude.com/docs/en/skills
- **How to create custom Skills**
  https://support.claude.com/en/articles/12512198
- **Anthropic engineering blog** — "Equipping agents for the real world with Agent Skills"
  https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills
- **Public skills repository**
  https://github.com/anthropics/skills
- **Community skills list**
  https://github.com/travisvn/awesome-claude-skills

Thank you.

Find these slides at:

https://github.com/aspi6246/ClaudeCodeTools

Find Skills at:

https://github.com/aspi6246/Claude-Code-Skills-for-Academics